Docker Containers — Hands-on Assignment

AI-Driven Software Engineering & Project Management (24CS3502) · Unit 3 · Department of CSE, Dayananda Sagar University

Assignment details

Students do Docker in two ways. **Part A:** create an Ubuntu Linux virtual machine in VMware and run the Docker experiments inside it from the terminal. **Part B:** install the Docker Desktop application on their own laptop and manage containers through its graphical interface. Every required step is proved with a screenshot in a report submitted as a single PDF file.

Item	Detail
Course	AI-Driven Software Engineering & Project Management (24CS3502)
Unit	Unit 3 — Containerization with Docker
Topic	Creating, packaging, deploying and managing applications with Docker
Mode	Individual
Part A	VMware + Ubuntu Linux VM + Docker Engine (command line) — Experiments A1–A11
Part B	Docker Desktop application on your own laptop (GUI) — Experiments B1–B5
Host laptop	Windows 10/11, macOS or Linux; minimum 8 GB RAM and 40 GB free disk recommended
Maximum marks	20
Submission	One PDF file, uploaded to a Google Drive link shared by the instructor
Due date	The Drive link stays open for **2 days** after it is shared; the deadline is when the link closes

Learning outcomes

After completing this assignment, the student will be able to:

1. Explain what a container is and how it differs from a virtual machine.
2. Create a Linux virtual machine in VMware and install Docker Engine inside it.
3. Pull images, run containers, and manage their lifecycle (start, stop, remove) from the command line.
4. Expose a containerized web server on a host port and view it in a browser.
5. Write a Dockerfile, build a custom image, and run it.
6. Use volumes to persist data and Docker Compose to run a multi-container app.
7. Install Docker Desktop and manage images, containers, logs and volumes through its interface.

Screenshot rules (read carefully)

- **Part A screenshots** must show the VMware window frame with the VM name (`usn-vm`) visible, so it is clear the work was done inside the VM.
- Every terminal screenshot must show **your own username** in the prompt, or your name typed in a comment line (`# Name: ____ USN: ____`) just before the command.
- Use a terminal font size large enough that the text is readable in the PDF without zooming in.
- Crop to the relevant window; do not paste full-screen captures with taskbars and unrelated tabs.
- Paste screenshots as images into your report (no more than 4 per page). Do not take photos of the screen with a phone.
- Below each screenshot, write a 1–2 line caption in your own words explaining what it shows.
- How to take screenshots: Windows `Win + Shift + S` · macOS `Cmd + Shift + 4` · Ubuntu `PrtSc` or `Shift + PrtSc`.

Background: what you need to know first

A container packages an application together with everything it needs to run — code, runtime, libraries, settings — so it runs the same way on every machine.

Analogy. Think of a shipping container. Before standardized containers, every port loaded cargo differently. Now any crane, ship or truck can move any container without caring what is inside. Docker does the same for software: “it works on my machine” becomes “it works on every machine that has Docker.”

In Part A you will see both technologies at once: VMware runs a full **virtual machine** (a complete Ubuntu OS), and inside it Docker runs lightweight **containers**.

Containers vs virtual machines

Aspect	Virtual machine	Container
What is virtualized	Entire hardware; each VM runs a full guest OS	Only the user space; all containers share the host OS kernel
Size	Gigabytes	Megabytes (e.g. `alpine` image ≈ 8 MB)
Start-up time	Minutes	Seconds or less
Isolation	Strong (separate kernel)	Process-level (namespaces, cgroups)
Example tool	VMware Workstation, VirtualBox	Docker

Key terms

- **Hypervisor** — software that runs virtual machines, e.g. VMware Workstation.
- **Image** — a read-only template (like a class in OOP). Built in layers. Example: `nginx:latest`.
- **Container** — a running instance of an image (like an object created from a class). You can run many containers from one image.
- **Dockerfile** — a text file of instructions (`FROM`, `COPY`, `RUN`, `CMD`) used to build an image.
- **Registry** — a store of images. **Docker Hub** (hub.docker.com) is the default public registry.
- **Volume** — storage that lives outside the container, so data survives when the container is deleted.
- **Port mapping** — `-p 8080:80` connects port 8080 on the machine to port 80 inside the container.
- **Docker Compose** — a YAML file (`compose.yaml`) that defines and runs several containers together.
- **Docker Engine** — the Docker background service and command-line tool; this is what you install in the Ubuntu VM.
- **Docker Desktop** — a graphical application for Windows, macOS and Linux that bundles Docker Engine, CLI, Compose and a dashboard to view images, containers and volumes.

The basic workflow

1. Write a `Dockerfile`.
2. `docker build` → creates an **image**.
3. `docker push` → stores the image in a **registry** (optional).
4. `docker pull` + `docker run` → creates a **container** on any machine.

Part A — Docker inside an Ubuntu virtual machine (VMware)

Perform Experiments A1–A11 in order. A1 and A2 are done once; A3–A11 are run **inside the Ubuntu VM's terminal**. Each experiment lists the commands and the screenshots (marked **SS**) to include.

In every command, replace `usn` with your own USN in lowercase (e.g. `1ds24cs001`).

Experiment A1 — Install VMware and create an Ubuntu VM

Aim: Create a Linux virtual machine on your laptop.

1. **Enable virtualization** in BIOS/UEFI if it is off (Intel VT-x or AMD-V). On Windows, check Task Manager → Performance → CPU → "Virtualization: Enabled".
2. **Install VMware** (free for personal use):
   - Windows / Linux host: VMware Workstation Pro.
   - macOS host: VMware Fusion. On Apple Silicon Macs (M1–M4) use the **ARM64** Ubuntu ISO.
   - Download from the Broadcom support portal (free account required). If you cannot get VMware, use Oracle VirtualBox and mention this in your report.
3. **Download Ubuntu 24.04 LTS Desktop** ISO from ubuntu.com/download.
4. **Create a new VM** in VMware: *Create a New Virtual Machine* → select the ISO → name it `usn-vm` → settings: **4 GB RAM, 2 CPU cores, 25 GB disk**, network **NAT**.
5. Power on the VM and complete the Ubuntu installation. Create a user account with your name.
6. Inside Ubuntu, open **Terminal** and check the system:

```
hostnamectl
lsb_release -a
free -h
nproc
```

- **SS A1.1** VMware VM settings screen showing name `usn-vm`, memory, processors and disk.
- **SS A1.2** Ubuntu desktop running inside VMware with the terminal output of `lsb_release -a` and `free -h`.

Experiment A2 — Install Docker Engine inside the VM

Aim: Install Docker from Docker's official repository and confirm it runs.

Run these commands in the Ubuntu VM terminal, one at a time:

```
sudo apt update
sudo apt install -y ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc
echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.asc] \
  https://download.docker.com/linux/ubuntu $(. /etc/os-release && echo $VERSION_CODENAME) stable" \
  | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin
```

Allow your user to run Docker without `sudo`, then **log out and log back in** (or restart the VM):

```
sudo usermod -aG docker $USER
```

Verify:

```
docker --version
docker version
docker compose version
systemctl status docker
```

- **SS A2.1** Final lines of the `apt install` output showing Docker packages installed.
- **SS A2.2** Output of `docker version` (both Client and Server sections) and `systemctl status docker` showing *active (running)*.

Experiment A3 — Run your first container

Aim: Understand what happens when you `docker run` an image you do not have.

```
docker run hello-world
```

Read the output: Docker could not find the image locally, pulled it from Docker Hub, created a container, ran it, and the container exited.

- **SS A3.1** Full output of `docker run hello-world`.
- **Write:** List the 4 steps Docker says it performed, in your own words.

Experiment A4 — Working with images

Aim: Pull, list, inspect and compare images.

```
docker pull alpine
docker pull ubuntu:24.04
docker images
docker image inspect alpine
docker history ubuntu:24.04
```

- **SS A4.1** Output of `docker images` showing alpine, ubuntu and hello-world with their sizes.
- **SS A4.2** Output of `docker history ubuntu:24.04` showing the image layers.
- **Write:** Why is `alpine` so much smaller than `ubuntu`? Compare the size of the `ubuntu:24.04` image with the 25 GB disk you gave the VM.

Experiment A5 — Interactive container

Aim: Go inside a container and see that it is an isolated Linux environment.

```
docker run -it --name usn-ubuntu ubuntu:24.04 bash
```

Inside the container run:

```
cat /etc/os-release
hostname
ls /
ps aux
exit
```

Back in the VM terminal:

```
hostname
ps aux | wc -l
docker ps
docker ps -a
```

- **SS A5.1** Commands and output inside the container (prompt shows `root@<container-id>`).
- **SS A5.2** Output of `docker ps -a` showing `usn-ubuntu` with status *Exited*.
- **Write:** Compare the hostname and number of processes inside the container with those of the VM. Why does `docker ps` not show the container but `docker ps -a` does?

Experiment A6 — Container lifecycle

Aim: Run a container in the background and control it.

```
docker run -d --name usn-web nginx
docker ps
docker logs usn-web
docker stop usn-web
docker ps -a
docker start usn-web
docker restart usn-web
docker stats --no-stream
docker rm -f usn-web
```

- **SS A6.1** Output from `docker run -d` through `docker ps -a` (status changes from *Up* to *Exited*).
- **SS A6.2** Output of `docker stats --no-stream` showing CPU and memory usage.
- **Write:** What does the `-d` flag do? Draw the lifecycle: created → running → stopped → removed.

Experiment A7 — Port mapping: serve a web page

Aim: Access a web server running inside a container from a browser.

```
docker run -d -p 8080:80 --name usn-web nginx
docker ps
curl http://localhost:8080
```

Open **Firefox inside the Ubuntu VM** and go to **http://localhost:8080**.

- **SS A7.1** Firefox in the VM showing “Welcome to nginx!” with the address bar visible.
- **SS A7.2** Output of `docker ps` showing the `PORTS` column `0.0.0.0:8080->80/tcp`.
- **Write:** Explain `-p 8080:80` — which number is the VM’s port and which is the container’s port?

Experiment A8 — Modify a running container

Aim: Execute commands inside a running container and see the change live.

```
docker exec -it usn-web sh
```

Inside the container (replace the name and USN):

```
echo "<h1>Hello from Your Name, USN 1DS24CS001</h1>" > /usr/share/nginx/html/index.html
exit
```

Refresh http://localhost:8080. Then remove and recreate the container:

```
docker rm -f usn-web
docker run -d -p 8080:80 --name usn-web nginx
```

Refresh the browser again.

- **SS A8.1** Browser showing your custom page with your name.
- **SS A8.2** Browser after recreating the container (back to the default nginx page).
- **Write:** Your change disappeared. Why? What does this tell you about where container data lives? (Experiment A10 fixes this.)

Experiment A9 — Build your own image with a Dockerfile

Aim: Package a small Python web app as a Docker image.

In the VM, create a folder and open the files in the text editor:

```
mkdir ~/docker-lab && cd ~/docker-lab
gedit app.py Dockerfile &
```

`app.py`:

```
from http.server import BaseHTTPRequestHandler, HTTPServer
import socket

NAME = "Your Name"
USN = "1DS24CS001"

class Handler(BaseHTTPRequestHandler):
    def do_GET(self):
        self.send_response(200)
        self.send_header("Content-Type", "text/html")
        self.end_headers()
        msg = (f"<h1>Containerized app by {NAME} ({USN})</h1>"
               f"<p>Container hostname: {socket.gethostname()}</p>")
        self.wfile.write(msg.encode())

HTTPServer(("0.0.0.0", 5000), Handler).serve_forever()
```

`Dockerfile` (no file extension):

```
FROM python:3.12-slim
WORKDIR /app
COPY app.py .
EXPOSE 5000
CMD ["python", "app.py"]
```

Build and run from inside `~/docker-lab`:

```
docker build -t usn-pyapp:1.0 .
docker images
docker run -d -p 5000:5000 --name usn-pyapp usn-pyapp:1.0
```

Open **http://localhost:5000** in Firefox inside the VM.

- **SS A9.1** Your `Dockerfile` and `app.py` open in the editor.
- **SS A9.2** Output of `docker build` (showing the build steps) and `docker images` listing `usn-pyapp`.
- **SS A9.3** Browser showing your name and the container hostname.
- **Write:** Explain each of the 5 Dockerfile lines in one sentence. Compare the hostname in the browser with the container ID in `docker ps`.

Experiment A10 — Persist data with volumes

Aim: Store data that survives container deletion.

```
docker volume create usn-data
docker volume ls
docker run --rm -v usn-data:/data alpine sh -c "echo Saved by USN 1DS24CS001 > /data/note.txt"
docker run --rm -v usn-data:/data alpine cat /data/note.txt
docker volume inspect usn-data
sudo cat /var/lib/docker/volumes/usn-data/_data/note.txt
```

The first container wrote the file and was deleted (`--rm`). A brand-new container still reads it, and you can also find the file on the VM's own disk.

- **SS A10.1** All commands and output above, ending with your message read directly from `/var/lib/docker/volumes/...`.
- **Write:** Contrast this with Experiment A8. Give one real-world case where a volume is essential.

Experiment A11 — Multi-container app with Docker Compose, then clean up

Aim: Start several containers with one command, then free disk space.

```
mkdir ~/compose-lab && cd ~/compose-lab
gedit compose.yaml &
```

`compose.yaml`:

```
services:
  web:
    image: nginx
    ports:
      - "8081:80"
  cache:
    image: redis:alpine
```

Run from inside `~/compose-lab`:

```
docker compose up -d
docker compose ps
docker compose logs cache
docker compose exec cache redis-cli ping
docker compose down
```

Then clean up everything from Part A:

```
docker system df
docker rm -f usn-web usn-pyapp usn-ubuntu
docker container prune -f
docker image prune -a -f
docker system df
```

- **SS A11.1** Output of `docker compose up -d` and `docker compose ps` showing both services running.
- **SS A11.2** Output of `redis-cli ping` returning `PONG`.
- **SS A11.3** `docker system df` before and after cleanup.
- **Write:** What advantage does Compose give over running each container with `docker run`?

Part B — Docker Desktop application

Install Docker Desktop on your **own laptop (the host, not the VM)** and repeat key tasks using its graphical interface. Screenshots must show the Docker Desktop window.

Experiment B1 — Install Docker Desktop and verify

Aim: Install the Docker Desktop application and confirm the engine is running.

- **Windows 10/11:** open PowerShell as Administrator and run `wsl --install` (restart if asked). Download Docker Desktop from docs.docker.com/desktop and install with the "Use WSL 2" option ticked.
- **macOS:** download the Docker Desktop `.dmg` matching your chip (Apple Silicon or Intel), drag it to Applications and launch it.
- **Linux:** install Docker Desktop for Linux from docs.docker.com/desktop (Ubuntu `.deb` package).

Launch Docker Desktop, accept the terms (you may skip sign-in) and wait for **"Engine running"**. Then open a terminal on the laptop:

```
docker version
docker context ls
```

- **SS B1.1** Docker Desktop dashboard showing “Engine running” in the status bar.
- **SS B1.2** Host terminal output of `docker version` (Server line shows *Docker Desktop*).

Experiment B2 — Pull and run a container from the GUI

Aim: Run a web server without typing any `docker run` command.

1. In Docker Desktop, use the **search bar** at the top to search `nginx` → select the official image → **Pull**.
2. Go to **Images**, find `nginx`, click **Run (▶)** → expand **Optional settings**:
   - Container name: `usn-desktop-web`
   - Host port: `8090`
3. Click **Run**. Open **http://localhost:8090** in your laptop’s browser.

- **SS B2.1** The *Optional settings* dialog filled with your container name and port 8090.
- **SS B2.2** Browser on the laptop showing “Welcome to nginx!” at `localhost:8090`.

Experiment B3 — Inspect and manage a running container

Aim: Use Docker Desktop’s container tabs instead of CLI commands.

Go to **Containers** → click `usn-desktop-web`, then open each tab:

- **Logs** — refresh the browser page a few times and watch new log lines appear.
- **Inspect** — find the container’s IP address and port bindings.
- **Exec** (terminal) — run `hostname` and `cat /etc/os-release`.
- **Stats** — CPU and memory graphs.

Finally, use the **Stop**, **Start** and **Delete** buttons on the container.

- **SS B3.1** Logs tab showing the GET requests from your browser.
- **SS B3.2** Exec tab showing the output of `hostname` and `cat /etc/os-release`.
- **SS B3.3** Containers list showing `usn-desktop-web` with status *Exited* after you pressed Stop.
- **Write:** For each tab (Logs, Inspect, Exec, Stats), write the equivalent CLI command you used in Part A.

Experiment B4 — Images, volumes and Compose stacks in the GUI

Aim: See Compose projects and volumes managed visually.

1. On the laptop, create a folder `compose-lab` with the same `compose.yaml` as Experiment A11, then in that folder run:

```
docker compose up -d
docker volume create usn-desktop-data
```

2. In Docker Desktop, open **Containers** — the two services appear grouped under `compose-lab`.
3. Open **Images** and **Volumes** tabs.
4. Stop and delete the `compose-lab` group using the GUI buttons; delete unused images from the **Images** tab.

- **SS B4.1** Containers tab showing the `compose-lab` group expanded with both containers running.
- **SS B4.2** Images tab listing the pulled images with their sizes.
- **SS B4.3** Volumes tab showing `usn-desktop-data`.

Experiment B5 — Compare the two approaches

Aim: Reflect on VM-based Docker versus Docker Desktop.

Copy and complete this table in your report from your own experience:

Point of comparison	Part A: Docker Engine in VMware VM	Part B: Docker Desktop
Time taken to set up		
Disk space used		
RAM used while running (check Task Manager / Activity Monitor)		
How you ran containers		
How you viewed logs		
Which you would use for learning, and why		
Which you would use on a production server, and why		

> - **Write:** Docker Desktop on Windows and macOS also runs a small hidden Linux VM. Why does Docker need a Linux kernel, and how is this similar to what you did manually in Part A?

Bonus (+2 marks, optional) — Publish your image

From the Ubuntu VM, create a free Docker Hub account, then:

```
docker login
docker tag usn-pyapp:1.0 <dockerhub-username>/usn-pyapp:1.0
docker push <dockerhub-username>/usn-pyapp:1.0
```

On your laptop, pull and run it in Docker Desktop to show the same image runs on both machines.

> - **SS Bonus.1** Successful push output in the VM.
> - **SS Bonus.2** The same image running in Docker Desktop on the laptop.

Troubleshooting

Problem	Fix
VM will not start: “virtualization disabled” / VT-x error	Enable Intel VT-x or AMD-V in BIOS/UEFI; on Windows also enable “Virtual Machine Platform” in Windows Features
VM is very slow	Give it 4 GB RAM and 2 CPUs; close other apps; install VMware Tools (`sudo apt install open-vm-tools-desktop`)
No internet inside the VM	VM settings → Network Adapter → NAT; restart the VM
`Cannot connect to the Docker daemon`	VM: `sudo systemctl start docker`. Desktop: start Docker Desktop and wait for “Engine running”
`permission denied ... docker.sock` (VM)	Run `sudo usermod -aG docker $USER`, then log out and back in
WSL 2 error (Windows, Part B)	Run `wsl --update` in PowerShell as Administrator; restart
`port is already allocated`	Another container uses that port: `docker ps`, then remove it or use a different host port such as `-p 8095:80`
`Conflict. The container name ... is already in use`	`docker rm -f <name>` and run again
Very slow pull on campus Wi-Fi	Download the Ubuntu ISO and pull images at home in advance

Report format and evaluation

The PDF report is graded out of 20, mostly on whether every required screenshot is present, readable and explained in your own words.

What the report must contain (in this order)

1. **Cover page** — college name, department, course name and code, assignment title (“Docker Containers — Hands-on Assignment”), your name, USN, section, date of submission.
2. **System details** — one table: host OS and version, processor, RAM, VMware version, VM settings (RAM/CPU/disk), Ubuntu version in the VM, Docker version in the VM, Docker Desktop version.
3. **Part A** — one section per experiment (A1–A11): heading with experiment number and aim, the screenshots labelled exactly as in this sheet (SS A1.1, SS A1.2 …), a 1–2 line caption under each, and the answers to the **Write** questions.
4. **Part B** — the same for B1–B5, including the completed comparison table.
5. **Command summary** — a table of every Docker command you used with a one-line description.

6. **Conclusion** — half a page: what you learned and one problem you faced and how you fixed it.

Expected length: 15–22 pages.

How to submit

1. Prepare the report in Word, Google Docs or LibreOffice, with screenshots pasted in, then export it as **one PDF file** (File → Save as / Download → PDF).
2. Name the file `USN_Name_Docker.pdf`, for example `1DS24CS001_RahulKumar_Docker.pdf`.
3. Keep the file under **20 MB**. If it is larger, compress the images before exporting.
4. Open the PDF once before uploading and check that every page and screenshot is readable.
5. The instructor will share a **Google Drive link** later. Upload your PDF there. The link stays open for **2 days** after it is shared; submissions are not accepted after it closes.
6. Upload only the PDF. Word files, image files, ZIP files and links to your own Drive are not accepted.

Evaluation rubric

Criterion	Marks
A1–A2: VMware VM created, Docker Engine installed in Ubuntu	3
A3–A8: first container, images, interactive, lifecycle, ports, exec	4
A9–A11: Dockerfile, volumes, Compose and cleanup	4
B1–B4: Docker Desktop installation and GUI tasks	4
B5 comparison and answers to Write questions (own words, correct)	3
Report quality: readable screenshots, labels, captions, cover page	2
Total	**20**
Bonus: image pushed to Docker Hub and run on both machines	+2

Academic integrity

- Screenshots without your USN in the VM name, container names, page content or prompt will get **zero** for that experiment.
- Identical screenshots or answers across two reports will be treated as copying; both reports get zero.
- If your laptop cannot run VMware or Docker Desktop (e.g. less than 8 GB RAM, or virtualization blocked), inform the instructor as soon as possible so you can use a lab system.